Object Oriented Programming (CSEG1044)

Unit 02 – Lecture 01: Classes, Objects, and Encapsulation (Basics)

Tofik Ali

School of Computer Science, UPES Dehradun

February 27, 2026

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Learning Outcomes

- Identify fields vs methods and explain state vs behavior.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Learning Outcomes

- Identify fields vs methods and explain state vs behavior.
- Explain references and aliasing using a small example.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Learning Outcomes

- Identify fields vs methods and explain state vs behavior.
- Explain references and aliasing using a small example.
- Apply encapsulation using `private` fields and validation methods.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Unit 02: Classes, Inheritance, Packages and Interfaces

- Lecture focus: Classes, Objects, and Encapsulation (Basics)

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Today: Roadmap

- Class fundamentals: fields, methods, state vs behavior

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Today: Roadmap

- Class fundamentals: fields, methods, state vs behavior
- Objects and reference semantics; object diagrams (mental model)

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Today: Roadmap

- Class fundamentals: fields, methods, state vs behavior
- Objects and reference semantics; object diagrams (mental model)
- Encapsulation motivation (invariants) + intro to access modifiers

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Class vs Object (quick)

- **Class** = blueprint (type): fields + methods

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Class vs Object (quick)

- **Class** = blueprint (type): fields + methods
- **Object** = instance at runtime (identity + state + behavior)

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Class vs Object (quick)

- **Class** = blueprint (type): fields + methods
- **Object** = instance at runtime (identity + state + behavior)
- State = values in fields; Behavior = what methods do

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

References and aliasing (important)

- In Java, variables of class type hold **references**.

Why it matters

Many bugs come from unexpected shared updates.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

References and aliasing (important)

- In Java, variables of class type hold **references**.
- Assignment copies the reference (not the whole object).

Why it matters

Many bugs come from unexpected shared updates.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

References and aliasing (important)

- In Java, variables of class type hold **references**.
- Assignment copies the reference (not the whole object).
- If two references point to the same object, changes are shared (**aliasing**).

Why it matters

Many bugs come from unexpected shared updates.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Encapsulation = protect invariants

- Make fields `private`.

Common mistake

Adding setters for everything and allowing invalid values.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Encapsulation = protect invariants

- Make fields `private`.
- Update state via methods that validate inputs.

Common mistake

Adding setters for everything and allowing invalid values.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Encapsulation = protect invariants

- Make fields `private`.
- Update state via methods that validate inputs.
- Keep object invariants always true (e.g., `0 <= cgpa <= 10`).

Common mistake

Adding setters for everything and allowing invalid values.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Example: Student (encapsulation) (1/2)

```java
public class Student {
    private final String rollNo;
    private final String name;
    private double cgpa; // invariant: 0..10

    public Student(String rollNo, String name, double cgpa
        ) {
        if (cgpa < 0.0 || cgpa > 10.0)
            throw new IllegalArgumentException("cgpa");
        this.rollNo = rollNo;
        this.name = name;
        this.cgpa = cgpa;
    }
```

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Example: Student (encapsulation) (2/2)

```
// continued...

    public void updateCgpa(double newCgpa) {
        if (newCgpa < 0.0 || newCgpa > 10.0)
            throw new IllegalArgumentException("cgpa");
        cgpa = newCgpa;
    }

    public double getCgpa() { return cgpa; }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Aliasing demo (observe)

```
public class Demo {
    public static void main(String[] args) {
        Student a = new Student("21CS1001", "Asha", 8.2);
        Student b = a; // same object
        b.updateCgpa(9.0);
        System.out.println(a.getCgpa()); // prints 9.0
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. What is the difference between a class and an object?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. What is the difference between a class and an object?
2. If `b = a` and `b` modifies the object, does `a` see it?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Mini Exercises (2–3 mins)

1. What is the difference between a class and an object?
2. If `b = a` and `b` modifies the object, does `a` see it?
3. Give one invariant for `BankAccount` or `LibraryBook`.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Key Takeaways

- Classes define state + behavior; objects are runtime instances.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Key Takeaways

- Classes define state + behavior; objects are runtime instances.
- Java object variables are references; aliasing is common.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Key Takeaways

- Classes define state + behavior; objects are runtime instances.
- Java object variables are references; aliasing is common.
- Encapsulation protects invariants using `private` + validation methods.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Checkpoint Questions

1. Why do we keep fields `private`?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Checkpoint Questions

1. Why do we keep fields `private`?
2. What is aliasing and why can it be risky?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Checkpoint Questions

1. Why do we keep fields `private`?
2. What is aliasing and why can it be risky?
3. What is an invariant? Give one example.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Next Lecture Preview

Next: Constructors, `this`, Overloading, `static`, and GC overview

- We will learn how objects are created and how constructors enforce invariants.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

Next Lecture Preview

Next: Constructors, `this`, Overloading, `static`, and GC overview

- We will learn how objects are created and how constructors enforce invariants.
- We will also learn when to use `static`.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).